

BÁO CÁO CHI TIẾT PHÂN TÍCH VÀ BẢO VỆ BACKEND LIFEBALANCE

Kiến trúc, thiết kế, vận hành, cách viết code và review từng module

Dự án	LifeBalance Backend
Loại tài liệu	Báo cáo kỹ thuật chi tiết phục vụ bảo vệ đồ án
Phạm vi	Toàn bộ backend gồm gateway, discovery, common/security và 8 service nghiệp vụ
Cấp độ giải thích	Từ kiến trúc tổng thể đến controller, service, repository, database, bảo mật, vận hành và lỗi cần review
Cách dùng	Đọc theo thứ tự nếu mới học; dùng phần câu hỏi phản biện để luyện bảo vệ
Vị trí lưu	E:\lifebalance-be\lifebalance-backend\Bao_cao_Bao_ve_Backend_LifeBalance_CHI_TIET.docx

Tài liệu này được viết theo hướng giúp người bảo vệ không chỉ nhớ tên module mà hiểu được vì sao backend được tách như vậy, một request thực tế đi qua những lớp nào, dữ liệu nằm ở đâu, bảo mật được kiểm tra ở đâu, và khi hội đồng hỏi sâu thì có thể trả lời bằng lập luận kỹ thuật.

Điểm quan trọng: hệ thống này không phải một ứng dụng Spring Boot đơn lẻ. Parent Maven quản lý 12 module. Trong đó có các module hạ tầng, module dùng chung và các service nghiệp vụ độc lập. Docker Compose vận hành các service này cùng PostgreSQL, Keycloak, Eureka, Gateway và các công cụ tùy chọn như RabbitMQ, Redis, MinIO, Prometheus, Grafana.

1. Ma Trận Yêu Cầu Của Báo Cáo

Phần này dùng để đối chiếu trực tiếp với yêu cầu bảo vệ. Khi nộp hoặc trình bày, có thể nói rằng báo cáo không chỉ mô tả hệ thống mà còn trả lời từng nhóm câu hỏi: kiến trúc đang dùng là gì, có bao nhiêu module, mỗi module làm gì, code được viết theo lớp nào, hệ thống vận hành thế nào, lỗi nào cần chú ý và nên bảo vệ ra sao.

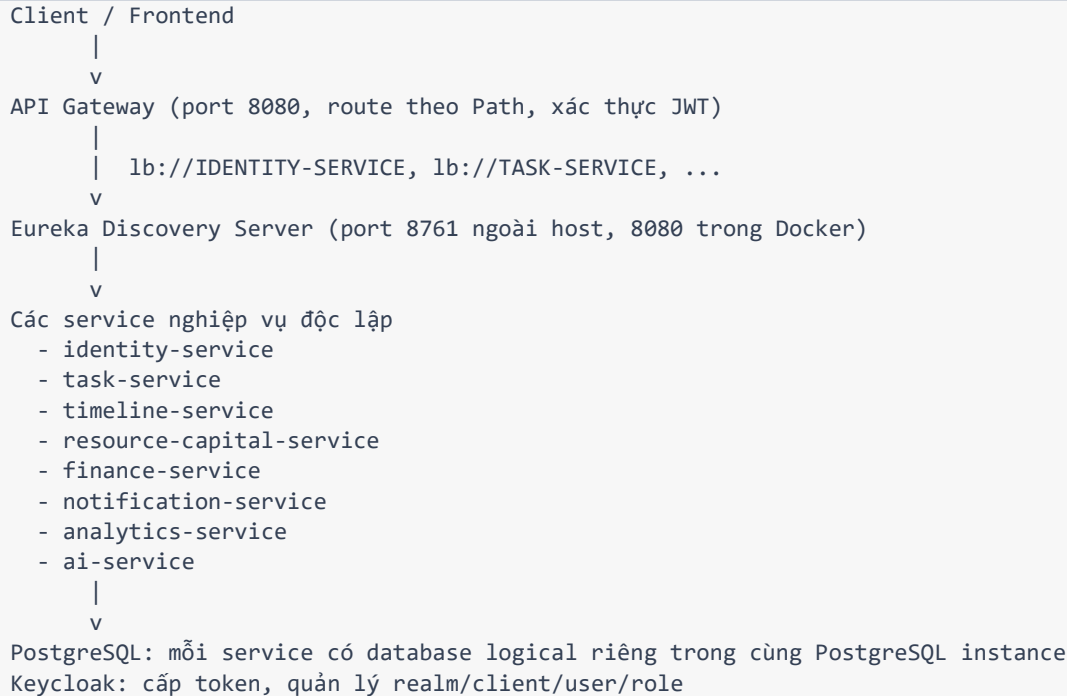
Yêu cầu cần đáp ứng	Nơi được giải thích trong báo cáo	Ý nghĩa khi bảo vệ
Backend đang dùng kiến trúc gì?	Chương 2, 3, 4	Giúp trả lời vì sao gọi đây là microservice-oriented backend chứ không phải monolithic backend.
Có bao nhiêu module chính?	Chương 3 và Chương 13-24	Nắm được 12 module Maven, chia thành hạ tầng, dùng chung và nghiệp vụ.
Discovery-server được xây dựng thế nào?	Chương 14	Giải thích Eureka Server, service registry, self-preservation, health, lỗi hostname đã được sửa.
Gateway được xây dựng thế nào?	Chương 15	Giải thích API Gateway, route theo Path, lb:// service name, security, actuator, hạn chế cần review.
Cách thiết kế database?	Chương 8 và từng module	Phân biệt database per service ở mức logical DB trong cùng PostgreSQL instance.
Cách vận hành backend?	Chương 11	Biết thứ tự chạy Docker, Keycloak, Eureka, Gateway, service và healthcheck.
Cách viết code?	Chương 6, 7, 9, 10	Giải thích Controller-Service-Repository, DTO, validation, error handling, transaction.
Công dụng tất cả module?	Chương 13-24	Mỗi module có vai trò, API, dữ liệu, cách chạy và checklist review.
Có cần sửa lỗi không?	Chương 25	Tách lỗi đã sửa, lỗi còn cần theo dõi, lỗi ảnh hưởng lớn và lỗi ảnh hưởng vừa.
Chuẩn bị phản biện	Chương 26	Có câu hỏi hội đồng thường hỏi và mẫu trả lời đủ sâu.

2. Kết Luận Kiến Trúc Tổng Thể

Backend LifeBalance được xây dựng theo hướng microservice. Nói chính xác hơn, đây là một hệ thống nhiều Spring Boot service độc lập được đóng gói trong một Maven multi-module project và vận hành bằng Docker Compose. Các service nghiệp vụ không nằm trong cùng một ứng dụng runtime; mỗi service có controller, service layer, repository, entity, cấu hình datasource và cổng debug riêng. Phía trước các service có API Gateway. Việc tìm service không viết cứng bằng host cố định mà đi qua Eureka Discovery Server với URI dạng lb://SERVICE-NAME.

Khi bảo vệ, không nên nói đơn giản là “em dùng microservice” rồi dừng ở đó. Cần giải thích đủ ba lớp. Lớp build-time là Maven parent quản lý nhiều module. Lớp runtime là nhiều container hoặc nhiều process Spring Boot chạy độc lập. Lớp giao tiếp là gateway định tuyến client request, Eureka quản lý service registry, các service gọi nhau qua HTTP nội bộ khi cần đồng bộ nghiệp vụ.

Hệ thống chưa phải một nền tảng cloud-native hoàn chỉnh ở mức production lớn, vì chưa thể hiện đầy đủ các thành phần như distributed tracing bắt buộc, circuit breaker nhất quán, event broker dùng bắt buộc cho tất cả tác vụ bất đồng bộ, hay autoscaling trên Kubernetes. Nhưng với phạm vi đồ án, kiến trúc đã có các đặc trưng cốt lõi của microservice: tách bounded context, tách module triển khai, tách database theo service ở mức logical database, có service discovery, có gateway, có security dùng chung và có health/monitoring endpoint.



Điểm dễ bị hỏi nhất là “database riêng” nghĩa là gì. Trong Compose, các biến môi trường như IDENTITY_DB, TASK_DB, TIMELINE_DB, FINANCE_DB... cho thấy mỗi service kết nối đến một database logical riêng. Tuy nhiên các database này đang cùng nằm trong một PostgreSQL container. Vì vậy khi trình bày cần nói rõ: dữ liệu được tách theo service ở mức schema/database ownership, còn hạ tầng PostgreSQL vật lý vẫn dùng chung để đơn giản hóa triển khai đồ án.

3. Backend Được Xây Dựng Từ Bao Nhiêu Phần Chính?

Parent Maven của dự án khai báo 12 module. Có thể chia thành 3 nhóm lớn để dễ bảo vệ: nhóm hạ tầng điều phối, nhóm dùng chung, và nhóm service nghiệp vụ. Cách chia này quan trọng vì nó giúp hội đồng thấy backend không phải tách file tùy tiện, mà tách theo trách nhiệm.

Nhóm	Module	Vai trò ngắn gọn
Hạ tầng điều phối	discovery-server	Eureka Server, nơi các service đăng ký và nơi gateway tra cứu instance.
Hạ tầng điều phối	gateway	Cửa vào duy nhất cho client, định tuyến request đến service tương ứng và áp dụng bảo mật chung.
Dùng chung	lifebalance-common	Chuẩn hóa response, error, exception, pageable limit, status response.

Nhóm	Module	Vai trò ngắn gọn
Dùng chung	lifebalance-security	Auto-configuration cho Spring Security, JWT, Keycloak principal, handler 401/403.
Nghiệp vụ	identity-service	Quản lý người dùng, vai trò, quyền, đăng nhập/xác thực, audit, hỗ trợ quản trị.
Nghiệp vụ	task-service	Quản lý task, category, tag, reminder, recurring rule, lịch sử thay đổi task.
Nghiệp vụ	timeline-service	Quản lý timeline, placement, lịch ngày/tuần, availability và snapshot task phục vụ lập lịch.
Nghiệp vụ	resource-capital-service	Quản lý vốn thời gian/tiền, chu kỳ vốn, phân bổ, điều chỉnh, giải phóng, tái phân bổ.
Nghiệp vụ	finance-service	Quản lý tài khoản tài chính, giao dịch, ngân sách, category tài chính, báo cáo.
Nghiệp vụ	notification-service	Quản lý thông báo, template, preference, delivery attempt, lịch sử gửi.
Nghiệp vụ	analytics-service	Quản lý dashboard, report, tracking/evaluation, actual records, lịch sử phân tích.
Nghiệp vụ	ai-service	Quản lý recommendation, insight, conversation, message, lịch sử AI.

Khi bị hỏi “phần backend này được xây dựng từ bao nhiêu phần chính”, câu trả lời gọn là: có 12 module Maven. Nếu chia theo chức năng hệ thống thì có 4 module nền tảng/dùng chung và 8 service nghiệp vụ. Nếu chia theo runtime thì có gateway, discovery-server, 8 service nghiệp vụ, PostgreSQL, Keycloak và các service phụ trợ tùy chọn như monitoring/cache/messaging/storage.

4. Vì Sao Không Viết Một Backend Duy Nhất?

Nếu viết một backend duy nhất, tất cả nghiệp vụ người dùng, task, timeline, tài chính, thông báo, phân tích và AI sẽ nằm trong cùng một codebase runtime. Cách đó đơn giản lúc đầu nhưng về lâu dài gây khó bảo trì: thay đổi ở finance có thể ảnh hưởng task, lỗi ở notification có thể làm cả backend lỗi, database dễ thành một khối quan hệ chằng chịt, và team khó chia việc theo domain.

LifeBalance có nhiều miền nghiệp vụ tương đối độc lập. Identity chịu trách nhiệm danh tính và phân quyền. Task chịu trách nhiệm việc cần làm. Timeline chịu trách nhiệm lịch và placement. Resource Capital chịu trách nhiệm phân bổ vốn thời gian/tiền. Finance chịu trách nhiệm giao dịch và ngân sách. Notification chịu trách nhiệm gửi thông báo. Analytics chịu trách nhiệm số liệu và báo cáo. AI chịu trách nhiệm gợi ý và hội thoại thông minh. Các miền này liên quan với nhau nhưng không nên trộn toàn bộ logic vào một service.

Ưu điểm của cách tách hiện tại là mỗi service có ranh giới rõ hơn. Controller của task không cần biết chi tiết bảng finance. Finance không cần quản lý bảng users. Timeline có thể giữ snapshot task cần thiết cho lập lịch thay vì phụ thuộc trực tiếp vào toàn bộ database task. Khi một service thay đổi, chỉ cần review hợp đồng API và tác động liên service thay vì dò toàn bộ backend.

Nhược điểm là hệ thống phức tạp hơn monolith. Cần service discovery, gateway, cấu hình Docker, biến môi trường, token, internal secret, healthcheck và chiến lược xử lý lỗi khi service gọi nhau thất bại. Vì vậy khi bảo vệ, nên thừa nhận trade-off: microservice giúp tách trách nhiệm và mở rộng nhưng đổi lại cần xử lý giao tiếp mạng, consistency và vận hành phức tạp hơn.

- Nếu hội đồng hỏi vì sao tách identity: vì danh tính, xác thực, phân quyền là cross-cutting concern và có thể dùng cho mọi service.
- Nếu hỏi vì sao task và timeline tách nhau: task là dữ liệu công việc, timeline là cách sắp xếp công việc theo thời gian; hai domain liên quan nhưng không giống nhau.
- Nếu hỏi vì sao finance và resource-capital tách nhau: finance quản lý tiền/giao dịch, resource-capital quản lý khái niệm vốn tổng thể gồm cả thời gian và tiền.
- Nếu hỏi vì sao vẫn cùng PostgreSQL container: vì đồ án ưu tiên để chạy cục bộ; ranh giới service vẫn thể hiện qua database logical và code ownership.

5. Luồng Request Thực Tế: Tạo Một Task

Đây là phần nên học kỹ nhất để bảo vệ, vì nó nối toàn bộ kiến trúc với code. Ví dụ người dùng tạo một task từ frontend. Frontend không gọi trực tiếp task-service bằng host nội bộ. Frontend gọi Gateway qua endpoint như /api/tasks. Gateway kiểm tra route Path và chuyển request đến TASK-SERVICE thông qua service discovery.

Trước khi vào controller nghiệp vụ, request đi qua Spring Security. Token JWT từ Keycloak được kiểm tra chữ ký, issuer, thời hạn, audience. Sau đó KeycloakUserMappingFilter ánh xạ thông tin người dùng trong token thành principal nội bộ như userId, username, roles/permissions. Controller không nên tin ownerId từ request body, mà lấy owner từ current user trong security context/request attribute.

```
Bước 1: Frontend gửi HTTP request
POST /api/tasks
Authorization: Bearer <access_token>
Body: { title, description, categoryId, dueDate, priority, ... }
```

```
Bước 2: Gateway nhận request
- So khớp Path=/api/tasks/**
- Route đến lb://TASK-SERVICE
- lb:// nghĩa là dùng load balancer + Eureka thay vì host cứng
```

```
Bước 3: task-service xử lý security
- Decode và validate JWT
- Map user từ Keycloak token sang KeycloakUserPrincipal
- Nếu token thiếu/sai/hết hạn: 401
- Nếu token hợp lệ nhưng không đủ quyền: 403
```

```
Bước 4: Controller nhận request
- Deserialize JSON thành CreateTaskRequest
- @Valid kiểm traNotBlank, NotNull, size, enum...
- Lấy ownerId từ currentUser
- Gọi TaskService.create(...)
```

```
Bước 5: Service xử lý nghiệp vụ
- Kiểm tra category/tag thuộc đúng owner
- Chuẩn hóa dữ liệu
- Tạo entity Task
- Ghi lịch sử thay đổi
- Chạy trong transaction cục bộ
```

```
Bước 6: Repository lưu database
- Spring Data JPA sinh SQL
```

- Commit nếu không lỗi
- Rollback nếu exception trong transaction

Bước 7: Side effects liên service

- Đồng bộ task sang timeline-service bằng internal API
- Tạo notification nếu cần
- Seed actual record cho analytics nếu cần

Bước 8: Response trả về frontend

- ApiResponse.success(data)
- Gateway chuyển response về client

Trong luồng này, điều cần nhấn mạnh là core transaction và side effect không giống nhau. Việc tạo task trong task-service là nghiệp vụ chính. Việc đồng bộ timeline, notification hoặc analytics là tác vụ liên service. Trong code hiện tại, các call liên service dùng RestClient và thường log warning khi thất bại. Cách này giúp task không nhất thiết thất bại chỉ vì notification-service tạm lỗi, nhưng cũng tạo rủi ro eventual consistency: task đã tạo nhưng timeline hoặc notification chưa kịp đồng bộ.

Nếu muốn nâng cấp production, nên dùng event/outbox pattern: task-service lưu task và lưu event trong cùng transaction, sau đó worker hoặc message broker gửi event cho timeline/notification/analytics. Như vậy không mất sự kiện khi service đích tạm thời lỗi. Với đồ án, có thể bảo vệ rằng phiên bản hiện tại chọn REST đồng bộ để dễ hiểu, dễ debug và phù hợp phạm vi, còn phần cải tiến đã nhận diện rõ.

6. Cách Viết Code Trong Mỗi Service

Các service nghiệp vụ nhìn chung đi theo layered architecture. Nghĩa là code không gom hết vào controller mà tách thành các lớp có trách nhiệm khác nhau. Tách lớp giúp đọc code nhanh hơn, test dễ hơn và giảm lỗi khi thay đổi nghiệp vụ. Một request thường đi từ Controller đến Service, từ Service đến Repository, từ Repository đến Database, rồi quay ngược lại thành DTO response.

Controller là lớp tiếp nhận HTTP. Nó định nghĩa URL, HTTP method, request body, path variable, query param, response status. Controller nên mỏng. Nó không nên chứa logic nghiệp vụ phức tạp như tính số dư vốn, kiểm tra allocation có vượt budget không, hay tự query nhiều repository. Controller chỉ nên validate đầu vào ở mức giao thức, lấy current user, gọi service và bọc response.

DTO là lớp dữ liệu đi qua API. Request DTO dùng để nhận input từ client. Response DTO dùng để trả output cho client. Không nên trả trực tiếp entity JPA ra ngoài, vì entity có quan hệ lazy, trường nội bộ, audit field hoặc cấu trúc database không nên lộ ra API. DTO cũng giúp API ổn định hơn ngay cả khi database entity thay đổi.

Service là nơi chứa nghiệp vụ. Service quyết định user có được thao tác không, trạng thái nào hợp lệ, khi nào tạo lịch sử, khi nào gọi integration client, khi nào transaction commit/rollback. Đây là phần quan trọng nhất khi review. Nếu logic nghiệp vụ bị đặt trong controller hoặc repository thì code sẽ khó test và dễ trùng lặp.

Repository là lớp truy cập dữ liệu. Dự án dùng Spring Data JPA nên repository thường khai báo interface, method query hoặc specification. Repository không nên quyết định nghiệp vụ. Ví dụ repository có thể tìm task theo ownerId và taskId, nhưng việc task đó có được chuyển trạng thái hay không phải do service quyết định.

Entity/domain model là ánh xạ database. Entity có annotation JPA, table name, column, quan hệ, audit field. Đây là nơi mô tả dữ liệu lưu trữ. Tuy nhiên entity không nên trở thành “thùng chứa mọi logic”. Với các nghiệp vụ có quy tắc rõ, có thể đặt invariant nhỏ trong domain method, còn orchestration lớn vẫn nên ở service.

Lớp code	Có nhiệm vụ gì	Không nên làm gì	Câu nói khi bảo vệ
Controller	Nhận HTTP request, mapping URL, validate DTO, lấy current user, gọi service.	Không viết business logic dài, không query DB trực tiếp, không tự xử lý transaction.	Controller là adapter từ HTTP vào use case.
DTO	Định nghĩa dữ liệu API vào/ra, validation annotation, tách API khỏi entity.	Không chứa logic truy cập DB, không thay thế entity.	DTO giúp API ổn định và tránh lộ cấu trúc database.
Service	Xử lý nghiệp vụ, phân quyền theo owner, transaction, lịch sử, orchestration.	Không phụ thuộc trực tiếp vào HTTP nếu không cần, không trả entity thô.	Service là trung tâm của use case.
Repository	Truy vấn/lưu entity qua JPA, cung cấp query theo owner/status/id.	Không quyết định quy tắc nghiệp vụ.	Repository là cổng vào database.
Integration client	Gọi service khác qua REST nội bộ, truyền token/secret, xử lý lỗi mạng.	Không trộn logic nghiệp vụ chính vào client.	Integration client cô lập giao tiếp liên service.
Exception handler	Chuẩn hóa lỗi thành ApiResponse/ApiError.	Không nuốt lỗi nghiêm trọng hoặc trả 200 cho lỗi.	Error handling giúp API nhất quán.

7. Thiết Kế API

API của backend đi theo hướng REST. URL đại diện cho tài nguyên, HTTP method đại diện cho hành động. GET dùng để đọc, POST để tạo, PUT/PATCH để cập nhật, DELETE để xóa hoặc hủy. Các controller trả response theo dạng ApiResponse để client nhận cùng một cấu trúc success/data/error/timestamp.

Một API tốt không chỉ có URL đúng mà còn phải rõ input, output và lỗi. Ví dụ POST /api/tasks cần nói rõ body gồm trường nào, trường nào bắt buộc, title có được rỗng không, categoryId có phải thuộc user hiện tại không, ngày đến hạn có hợp lệ không, khi tạo thành công trả về status gì, khi category không tồn tại trả 404 hay 400, khi task trùng logic trả 409 hay không.

Path variable dùng khi cần định danh một tài nguyên cụ thể, ví dụ /api/tasks/{taskId}. Query param dùng cho lọc, sắp xếp, phân trang, ví dụ page, size, status, from, to. Request body dùng cho dữ liệu tạo/cập nhật phức tạp. Header dùng cho thông tin giao thức như Authorization hoặc internal secret giữa các service.

Khái niệm API	Ví dụ trong backend	Ý nghĩa review
Resource path	/api/tasks, /api/timeline, /api/finance/accounts	Tên path phải ổn định, dễ hiểu, không trùng route giữa service.
Path variable	/api/tasks/{taskId}	Dùng cho tài nguyên cụ thể; service phải kiểm tra ownerId để tránh đọc dữ liệu user khác.
Query param	?page=0&size=20&status=OPEN	Cần giới hạn size bằng PageableLimits để tránh truy vấn quá lớn.
Request body	CreateTaskRequest, CreateBudgetRequest	Cần @Valid và thông báo lỗi rõ ràng.
Response wrapper	ApiResponse.success(data), ApiResponse.failure(error)	Client xử lý thống nhất, dễ debug.

Khái niệm API	Ví dụ trong backend	Ý nghĩa review
Error code	AUTH_UNAUTHORIZED, AUTH_FORBIDDEN, CommonErrorCode...	Không chỉ trả message tự do; nên có code ổn định.

Trong review gateway, cần kiểm tra API route ở gateway có khớp controller thật không. Ví dụ gateway đang route resource-capital theo `/api/v1/capital/**`, `/api/v1/capital-cycles/**`, `/api/v1/capital-adjustments/**`, `/api/v1/capital-allocations/**`. Nếu frontend hoặc test gọi nhầm `/api/v1/capitals/summary` dạng plural “capitals” trong khi route/controller là “capital”, request có thể không đi đến service đúng và gây 404. Đây là lỗi không làm sập hệ thống nhưng ảnh hưởng trực tiếp API contract.

8. Thiết Kế Database Và Sở Hữu Dữ Liệu

Trong microservice, nguyên tắc quan trọng là service nào sở hữu dữ liệu của service đó. Không nên để service A truy cập trực tiếp bảng của service B. Nếu resource-capital cần biết một task có thuộc user không, nó nên gọi task-service hoặc dùng một bản sao dữ liệu/sự kiện đã đồng bộ, thay vì query trực tiếp database task. Cách này giữ ranh giới domain rõ ràng.

Compose của dự án cấu hình mỗi service nghiệp vụ với datasource riêng: identity dùng IDENTITY_DB, task dùng TASK_DB, timeline dùng TIMELINE_DB, resource-capital dùng RESOURCE_CAPITAL_DB, finance dùng FINANCE_DB, notification dùng NOTIFICATION_DB, analytics dùng ANALYTICS_DB, ai dùng AI_DB. Các database này cùng nằm trong một PostgreSQL container để dễ chạy cục bộ, nhưng về mặt sở hữu dữ liệu, mỗi service có vùng dữ liệu riêng.

Khi bảo vệ, nên giải thích rằng khóa ngoại nội bộ trong cùng service là bình thường, ví dụ task có `task_tags`, `task_reminders`, `task_recurring_rules`. Nhưng khóa ngoại xuyên service thường nên tránh, vì nếu database của service khác thay đổi sẽ phá service hiện tại. Thay vì FK trực tiếp từ resource-capital sang task database, hệ thống dùng `targetId/taskId` và xác thực qua API task-service.

Service	Database logical	Nhóm bảng/domain chính	Ý nghĩa sở hữu dữ liệu
identity-service	IDENTITY_DB	users, roles, permissions, audit_logs, support_tickets, announcements, configs	Nguồn sự thật về người dùng, vai trò, quyền và quản trị.
task-service	TASK_DB	tasks, categories, tags, task_tags, reminders, recurring_rules, change_history	Nguồn sự thật về công việc và metadata task.
timeline-service	TIMELINE_DB	timeline_tasks, timeline_placements, timeline_histories	Nguồn sự thật về lịch, placement và snapshot phục vụ lập lịch.
resource-capital-service	RESOURCE_CAPITAL_DB	capital_cycles, time_capitals, money_capitals, allocations, adjustments, releases, reallocations	Nguồn sự thật về vốn thời gian/tiền và phân bổ.
finance-service	FINANCE_DB	finance_accounts, finance_categories, budgets, transactions, recurring_rules, histories	Nguồn sự thật về tiền, giao dịch, ngân sách.
notification-service	NOTIFICATION_DB	notifications, templates, preferences, delivery_attempts, histories	Nguồn sự thật về thông báo và lịch sử gửi.
analytics-service	ANALYTICS_DB	actual_records, reports, evaluation_results,	Nguồn sự thật về dữ liệu

Service	Database logical	Nhóm bảng/domain chính	Ý nghĩa sở hữu dữ liệu
		analytics_histories	phân tích, báo cáo, đánh giá.
ai-service	AI_DB	recommendations, insights, conversations, messages, histories	Nguồn sự thật về gợi ý, insight và hội thoại AI.

Review database cần nhìn các điểm: bảng có ownerId hoặc user reference đủ để cách ly dữ liệu người dùng không; truy vấn repository có filter theo ownerId không; các field tiền/thời gian dùng kiểu dữ liệu phù hợp không; các trạng thái quan trọng có enum rõ không; migration có version không; delete dữ liệu có cần soft delete/history không; và các truy vấn lọc/phân trang có index hợp lý không.

9. Bảo Mật: Authentication, Authorization, JWT, Keycloak

Authentication là xác minh người dùng là ai. Authorization là xác minh người đó được phép làm gì. Trong hệ thống này, Keycloak đóng vai trò identity provider: quản lý realm, user, client, role và cấp access token. Các service backend không tự lưu mật khẩu để xác thực trực tiếp trong từng service nghiệp vụ; thay vào đó chúng kiểm tra JWT do Keycloak cấp.

lifebalance-security là module rất quan trọng vì nó đóng gói cấu hình bảo mật dùng chung. Module này tạo SecurityFilterChain, tắt session state theo kiểu stateless, cấu hình OAuth2 Resource Server, dùng JwtDecoder để decode token, validate issuer/audience/thời hạn, ánh xạ token thành KeycloakUserPrincipal, và chuẩn hóa lỗi 401/403.

Trong cấu hình security, các endpoint health/info/prometheus/swagger/status/health được permit theo rule nhất định, còn các request khác yêu cầu authenticated. Điều này giúp service có thể được healthcheck bởi Docker/monitoring mà không cần token, trong khi API nghiệp vụ vẫn cần JWT. Đây là điểm bảo vệ quan trọng vì nếu health endpoint cũng cần token thì container orchestration khó kiểm tra readiness; nếu API nghiệp vụ public hết thì mất bảo mật.

JWT không chỉ là một chuỗi token. Service phải kiểm tra chữ ký qua JWK set, kiểm tra issuer đúng realm, kiểm tra token chưa hết hạn, và kiểm tra audience đúng client-id lifebalance-api. Nếu thiếu audience validation, token cấp cho client khác có thể bị dùng sai vào backend. Code hiện có JwtAudienceValidator.requireAudience(clientId), đây là điểm tốt khi review.

Trường hợp	HTTP status nên trả	Giải thích
Không gửi Authorization header	401 Unauthorized	Người dùng chưa được xác thực.
Token sai chữ ký/hết hạn/sai issuer	401 Unauthorized	Token không hợp lệ nên không xác định được identity đáng tin.
Token hợp lệ nhưng role/permission không đủ	403 Forbidden	Đã biết người dùng là ai nhưng không được phép làm hành động.
User A đọc dữ liệu User B	403 hoặc 404 tùy chính sách	Phải kiểm tra ownerId ở service, không tin id từ client.
Internal service gọi thiếu X-Lifebalance-Internal-Secret	403 Forbidden	Endpoint nội bộ cần thêm credential service-to-service.

Một lỗi bảo mật thường gặp là chỉ xác thực token nhưng quên kiểm tra ownership. Ví dụ user A có token hợp lệ, nhưng gọi GET /api/tasks/{taskId} với taskId của user B. Nếu service chỉ findById(taskId) mà không

findByIdAndOwnerId(taskId, ownerId), dữ liệu có thể bị lộ. Vì vậy khi review từng service, hãy tìm xem repository/service có filter theo ownerId không.

10. Validation, Error Handling Và Transaction

Validation là lớp phòng thủ đầu tiên để dữ liệu bản không đi sâu vào business logic. Request DTO nên dùng annotation như @NotNull, @NotBlank, @Size, @Min, @Max, @Valid. Tuy nhiên validation annotation không thay thế business validation. Ví dụ @NotNull chỉ biết categoryId không rỗng; nó không biết category đó có thuộc user hiện tại hay không. Việc đó phải do service kiểm tra.

Error handling giúp API trả lỗi thống nhất. lifebalance-common có ApiResponse, ApiError, AppException, error code và GlobalExceptionHandler. Lợi ích là frontend không phải đoán cấu trúc lỗi của từng service. Khi lỗi validation xảy ra, client nhận lỗi có code/message/details. Khi lỗi auth xảy ra, lifebalance-security trả AuthErrorCode như AUTH_UNAUTHORIZED hoặc AUTH_FORBIDDEN.

Transaction dùng để đảm bảo một nhóm thao tác database trong cùng service hoặc commit cùng nhau hoặc rollback cùng nhau. Ví dụ tạo capital allocation có thể cần ghi allocation, ghi history và cập nhật balance. Nếu một bước lỗi, transaction rollback để không có dữ liệu nửa vời. Nhưng transaction này là transaction cục bộ trong một service, không tự động bao trùm service khác.

Đây là điểm rất hay bị hỏi: “Nếu task-service tạo task xong nhưng timeline-service lỗi thì sao?”. Câu trả lời là local transaction của task-service đã commit, còn call sang timeline là giao tiếp liên service. Nếu muốn đảm bảo mạnh hơn thì dùng Saga/outbox/message broker. Phiên bản hiện tại chấp nhận eventual consistency cho side effect và log lỗi để không làm hỏng nghiệp vụ chính.

Loại lỗi	Ví dụ	Ảnh hưởng	Cách xử lý/review
Validation error	title rỗng, page size quá lớn	Request bị từ chối sớm	DTO annotation, PageableLimits, message rõ ràng.
Business error	allocation vượt vốn, category không thuộc user	Ngăn dữ liệu sai nghiệp vụ	Service check rõ và throw AppException có code.
Auth error	token thiếu/sai	Không cho truy cập API	Security filter trả 401.
Authorization error	user không đủ quyền hoặc truy cập dữ liệu người khác	Ngăn lộ dữ liệu/leo quyền	Owner check, role/permission check, 403/404.
Integration error	notification-service tạm down	Side effect mất/đến muộn	Log warning, retry/outbox nếu nâng cấp.
Infrastructure error	PostgreSQL down	Service không lưu/đọc được dữ liệu	Healthcheck, readiness, retry ở mức hạ tầng.

11. Cách Vận Hành Backend

Dự án được vận hành cục bộ chủ yếu bằng Docker Compose. Lệnh cơ bản là tạo file .env từ .env.example rồi chạy docker compose up -d --build. Docker sẽ build từng Spring Boot module thành image, chạy PostgreSQL, Keycloak, discovery-server, gateway và các service nghiệp vụ. Các service dùng network lifebalance-internal để gọi nhau bằng Docker DNS như postgres, keycloak, discovery-server.

Một lỗi vận hành hay gặp là dùng localhost bên trong container. Trong container, localhost là chính container đó chứ không phải máy host và cũng không phải container khác. Vì vậy datasource phải dùng

jdbc:postgresql://postgres:5432/..., Keycloak nội bộ phải là http://keycloak:8080, Eureka default zone là http://discovery-server:8080/eureka. Gateway ra ngoài host là localhost:8080, Eureka UI ngoài host là localhost:8761, nhưng nội bộ container lại dùng port 8080 theo service name.

Cách chạy cơ bản:
 Copy-Item .env.example .env
 docker compose up -d --build

URL ngoài máy host:
 PostgreSQL: localhost:5432
 Keycloak: localhost:8082
 Eureka: localhost:8761
 Gateway: localhost:8080

URL nội bộ container:
 postgres:5432
 keycloak:8080
 discovery-server:8080
 gateway:8080

Health endpoint:
 /actuator/health/readiness
 /actuator/health/liveness
 /actuator/prometheus

Compose còn có các profile tùy chọn: messaging cho RabbitMQ, cache cho Redis, storage cho MinIO, monitoring cho Prometheus/Grafana. Điều này cho thấy thiết kế đã nghĩ đến hạ tầng mở rộng, nhưng không phải mọi profile đều bắt buộc chạy trong flow lõi. Khi bảo vệ, nên nói rõ thành phần nào là bắt buộc để backend hoạt động và thành phần nào là tùy chọn để nâng cấp khả năng vận hành.

12. Testing Và Cách Review Backend

Review backend không chỉ là chạy thử thấy màn hình hoạt động. Cần kiểm tra code theo nhiều lớp: unit test cho service logic, controller test cho API và validation, repository test cho query/migration, security test cho 401/403, integration test cho luồng liên service nếu có, và smoke test bằng Docker để đảm bảo cấu hình runtime đúng.

Số lượng test hiện tại không đều giữa các service. Một số service có nhiều test hơn như identity-service, resource-capital-service, task-service; một số service ít test hơn như discovery-server, gateway, timeline, notification, ai. Điều này không nhất thiết là sai vì độ phức tạp domain khác nhau, nhưng khi review nên ưu tiên test cho nơi có logic nghiệp vụ nặng và nơi có rủi ro bảo mật/giao tiếp liên service.

Module	Số file Java	Số test đã ghi nhận	Nhận xét review
identity-service	179	43	Nhiều logic user/role/permission nên test tương đối quan trọng.
resource-capital-service	145	47	Domain phức tạp, test allocation/balance/history là điểm tốt.
task-service	89	19	Cần test task lifecycle, ownership, tag/category, integration side effect.

Module	Số file Java	Số test đã ghi nhận	Nhận xét review
finance-service	74	5	Nên bổ sung test transaction/budget/report nếu mở rộng.
analytics-service	61	8	Nên test aggregate/report/export kỹ hơn khi dữ liệu tăng.
notification-service	57	4	Nên test delivery retry/preference/template.
ai-service	55	5	Nên mock provider và test fallback/error path.
timeline-service	40	4	Nên test placement conflict, availability và task snapshot sync.
lifebalance-security	13	9	Test bảo mật dùng chung là quan trọng vì ảnh hưởng toàn hệ thống.
lifebalance-common	10	2	Ít nhưng module nhỏ; cần đảm bảo response/error ổn định.
gateway	1	3	Gateway ít Java code nhưng cấu hình route/security cần test chính xác.
discovery-server	1	2	Logic ít, chủ yếu test cấu hình profile/actuator.

Khi review gateway, cần kiểm tra các test về /actuator/prometheus và route path. Trước đó có dấu hiệu test kỳ vọng prometheus public nhưng thực tế trả 401 trong một ngữ cảnh test, và một test dùng path /api/v1/capitals/summary không khớp route /api/v1/capital/**. Những lỗi kiểu này ảnh hưởng lớn đến độ tin cậy test và API contract, nhưng không nhất thiết làm sập toàn bộ backend nếu client đang dùng đúng path.

Checklist review nên gồm: route gateway khớp controller; security public/private endpoint đúng; mỗi service filter theo ownerId; DTO validation đủ; transaction bọc đúng use case; lỗi trả code nhất quán; call liên service có timeout và xử lý lỗi; database migration chạy được; Docker env không để secret rỗng trong môi trường cần integration; healthcheck readiness phản ánh trạng thái thật.

13. Module lifebalance-common

Thông tin	Nội dung
Nhóm	Dùng chung
Đường dẫn chính	lifebalance-common/src/main/java/com/lifebalance/common
API/entry point	Không phải service runtime; được import bởi các service khác.
Dữ liệu sở hữu	Không sở hữu database nghiệp vụ.

lifebalance-common là module nền tảng để các service thống nhất cách trả response, cách mô tả lỗi và một số tiện ích web chung. Nếu không có module này, mỗi service có thể tự trả JSON khác nhau: service task trả {data, message}, service finance trả {success, result}, service identity trả lỗi theo format khác. Khi frontend tích hợp nhiều service qua gateway, sự không thống nhất đó làm code client rối và khó debug.

Thành phần quan trọng nhất là ApiResponse<T>. Cấu trúc gồm success, data, error, timestamp. Khi thành công, service dùng ApiResponse.success(data). Khi thất bại, service dùng ApiResponse.failure(error). Cách này giúp frontend chỉ cần nhìn success để biết request thành công hay không, nhìn data khi thành công và error khi thất bại. timestamp hỗ trợ debug vì biết lỗi xảy ra lúc nào.

ApiError chứa code, message và details. code dùng để máy xử lý ổn định, message dùng để người đọc hiểu, details dùng cho lỗi validation nhiều field. AuthErrorCode có các mã như AUTH_UNAUTHORIZED, AUTH_FORBIDDEN, AUTH_INVALID_TOKEN, AUTH_EXPIRED_TOKEN. CommonErrorCode và AppException giúp service ném lỗi nghiệp vụ có cấu trúc thay vì ném RuntimeException tùy tiện.

PageableLimits là chi tiết nhỏ nhưng có giá trị. Nếu client gửi size quá lớn, ví dụ size=100000, database có thể bị query nặng. PageableLimits chuẩn hóa pageable để giới hạn kích thước trang. Đây là kiểu defensive coding cần nhắc khi bảo vệ vì nó chứng minh hệ thống không chỉ chạy đúng mà còn chống lạm dụng input.

GlobalExceptionHandler giúp gom exception thành response chuẩn. Khi validation lỗi, khi AppException xảy ra, hoặc khi lỗi không mong muốn phát sinh, handler có thể trả status và body nhất quán. Đây là điểm làm API chuyên nghiệp hơn vì client không phải xử lý HTML error page hoặc JSON lỗi khác format.

Checklist review

- Kiểm tra service có dùng ApiResponse thống nhất không.
- Kiểm tra error code có đủ cụ thể để frontend xử lý không.
- Không nên đưa logic nghiệp vụ cụ thể của một service vào common, vì common phải giữ tính trung lập.
- Nếu common thay đổi API response, phải review toàn bộ service và frontend vì blast radius lớn.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Vì sao cần module common?	Để tránh lặp cấu trúc response/error ở từng service và giữ hợp đồng API nhất quán trên toàn backend.
Common có phải service không?	Không. Nó là thư viện nội bộ được các module khác import khi build, không chạy container riêng.

14. Module discovery-server

Thông tin	Nội dung
Nhóm	Hạ tầng điều phối
Đường dẫn chính	discovery-server
API/entry point	Eureka UI/registry; ngoài host thường là localhost:8761, trong Docker là discovery-server:8080.
Dữ liệu sở hữu	Không sở hữu database nghiệp vụ.

discovery-server được xây dựng bằng Spring Cloud Netflix Eureka Server. Vai trò của nó là service registry: các service như identity-service, task-service, finance-service khi khởi động sẽ đăng ký tên service, địa chỉ, port, health/status với Eureka. Gateway hoặc service khác không cần biết task-service đang chạy ở container nào; chúng chỉ cần gọi theo service name.

Trong application.yaml, discovery-server đặt spring.application.name là discovery-server. Eureka client của chính discovery-server được cấu hình register-with-eureka=false và fetch-registry=false. Lý do là nó là server registry, không cần tự đăng ký vào chính nó và không cần fetch registry từ server khác trong mô hình single Eureka server của đồ án.

Profile prod có server.port lấy từ DISCOVERY_PORT, eureka.instance.hostname lấy từ EUREKA_INSTANCE_HOSTNAME với default discovery-server, eureka.server.enable-self-preservation=true và

eviction interval. enable-self-preservation giúp Eureka tránh xóa hàng loạt service khi có vấn đề mạng tạm thời. Đây là cơ chế bảo vệ trong môi trường phân tán.

Lỗi từng cần sửa ở discovery-server là hostname trong môi trường Docker/production. Nếu Eureka quảng bá hostname không đúng, gateway có thể lấy registry nhưng gọi đến địa chỉ không truy cập được. Bản hiện tại đã cấu hình EUREKA_INSTANCE_HOSTNAME default là discovery-server và Compose truyền DISCOVERY_EUREKA_INSTANCE_HOSTNAME. Đây là hướng sửa hợp lý vì container trong Docker network có thể resolve discovery-server bằng DNS nội bộ.

Discovery-server không chứa business logic. Nếu nó down, các service đã có connection/routing cũ có thể còn hoạt động một thời gian tùy cache, nhưng service mới khởi động hoặc gateway cần cập nhật registry sẽ bị ảnh hưởng. Vì vậy discovery-server là thành phần hạ tầng quan trọng. Trong production lớn có thể chạy nhiều Eureka instance hoặc thay bằng service discovery của Kubernetes.

Checklist review

- Kiểm tra service name đăng ký có khớp lb://SERVICE-NAME trong gateway không.
- Kiểm tra hostname/port trong container không dùng localhost sai ngữ cảnh.
- Kiểm tra /actuator/health/readiness hoạt động để Docker biết service sẵn sàng.
- Nếu triển khai thật, cân nhắc nhiều instance discovery hoặc cơ chế discovery có HA.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Tại sao gateway không gọi thẳng task-service:8080?	Vì dùng service discovery giúp không phụ thuộc địa chỉ cứng và hỗ trợ nhiều instance/load balancing.
Discovery-server có xử lý request nghiệp vụ không?	Không. Nó chỉ quản lý registry, request nghiệp vụ đi qua gateway đến service tương ứng.

15. Module gateway

Thông tin	Nội dung
Nhóm	Hạ tầng điều phối
Đường dẫn chính	gateway
API/entry point	Cửa vào API: localhost:8080 ngoài host, gateway:8080 trong Docker.
Dữ liệu sở hữu	Không sở hữu database nghiệp vụ.

gateway là API Gateway của hệ thống. Nó được xây dựng bằng spring-cloud-starter-gateway-server-webmvc, Eureka Client, Actuator, Prometheus registry, OAuth2 Resource Server, Spring Security và lifebalance-security. Gateway có rất ít Java code vì vai trò chính nằm ở cấu hình route, security và integration với Spring Cloud.

Trong application.yaml, gateway định nghĩa các route bằng spring.cloud.gateway.server.webmvc.routes. Mỗi route có id, uri và predicate Path. uri có dạng lb://IDENTITY-SERVICE, lb://TASK-SERVICE... Chữ lb nghĩa là dùng load-balanced client, service instance lấy từ Eureka. Gateway không hard-code IP container của từng service.

Route identity nhận nhiều path vì identity-service có nhiều nhóm API: auth, users, roles, permissions, administration-support, audit-logs. Route task nhận /api/tasks/**, /api/categories/**, /api/tags/** và

/tags/**. Route resource-capital nhận /api/v1/capital/**, /api/v1/capital-cycles/**, /api/v1/capital-adjustments/**, /api/v1/capital-allocations/**. Các route còn lại lần lượt dành cho timeline, finance, notification, ai, analytics.

Gateway cũng là điểm thích hợp để đặt cross-cutting concerns: CORS, xác thực token, log request, metrics, rate limit, header forwarding. Trong dự án hiện tại, gateway dùng lifebalance-security để kiểm soát JWT và actuator/prometheus để quan sát. Gateway không nên chứa nghiệp vụ như tính ngân sách hay tạo task; nó chỉ là lớp vào và lớp định tuyến.

Điểm cần review kỹ ở gateway là route phải khớp controller. Nếu controller dùng /api/v1/capital nhưng test hoặc frontend gọi /api/v1/capitals, gateway có thể không route request. Điểm khác là endpoint /actuator/prometheus phải có chính sách bảo mật thống nhất với expectation của test và monitoring. Nếu monitoring cần scrape prometheus không token, rule security phải permit đúng trong profile runtime/test.

Checklist review

- So khớp tất cả route Path trong gateway với @RequestMapping của controller thật.
- Kiểm tra lb://SERVICE-NAME có khớp spring.application.name/service registration không.
- Kiểm tra security rule cho actuator/prometheus có đúng nhu cầu monitoring không.
- Không đưa business logic vào gateway.
- Nên bổ sung rate limit/correlation id nếu triển khai production.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Gateway khác discovery-server ở đâu?	Gateway nhận request từ client và định tuyến; discovery-server chỉ lưu registry để gateway biết service nào đang sống.
Gateway có thay thế Spring Security ở service con không?	Không nên. Gateway là lớp bảo vệ đầu vào, nhưng service con vẫn nên tự xác thực/ủy quyền vì có thể có call nội bộ hoặc cấu hình bypass gateway.

16. Module lifebalance-security

Thông tin	Nội dung
Nhóm	Dùng chung
Đường dẫn chính	lifebalance-security/src/main/java/com/lifebalance/security
API/entry point	Không có API nghiệp vụ riêng; auto-config cho các service import module này.
Dữ liệu sở hữu	Không sở hữu database nghiệp vụ.

lifebalance-security gom cấu hình bảo mật dùng chung để gateway và các service không phải viết lại SecurityFilterChain. Module này auto-config trước SecurityAutoConfiguration của Spring Boot, bật KeycloakSecurityProperties, tạo KeycloakUserMapper, JwtDecoder, AuthenticationFailureLogger, AuthorizationFailureLogger, BearerTokenResolver, AuthenticationEntryPoint, AccessDeniedHandler và SecurityFilterChain.

JwtDecoder được tạo lazy: khi token đầu tiên cần decode, code mới build NimbusJwtDecoder dựa trên issuer-uri hoặc jwk-set-uri. Decoder dùng DelegatingOAuth2TokenValidator để kết hợp validator mặc định của issuer/timestamp và JwtAudienceValidator.requireAudience(clientId). Điều này giúp token không chỉ đúng chữ ký mà còn đúng audience lifebalance-api.

SecurityFilterChain tắt CSRF, HTTP Basic, form login và dùng SessionCreationPolicy.STATELESS. Đây là cấu hình phù hợp API backend dùng JWT. Stateless nghĩa là server không lưu session người dùng; mỗi request phải mang token. Cách này phù hợp microservice vì service không phụ thuộc session memory local.

Module cũng permit một số endpoint công khai như /actuator/health/**, /actuator/info, /actuator/prometheus, swagger và GET status/health. Những endpoint còn lại yêu cầu authenticated. Sau khi OAuth2 resource server xác thực JWT, KeycloakUserMappingFilter chuyển token thành principal nội bộ. Nhờ vậy controller/service có thể lấy ownerId/currentUser thay vì tự parse JWT.

LifebalanceAuthenticationEntryPoint và LifebalanceAccessDeniedHandler chuẩn hóa response lỗi 401/403 bằng ApiError. Đây là điểm tốt vì lỗi bảo mật cũng cùng format với lỗi API khác. AuthenticationFailureLogger và AuthorizationFailureLogger hỗ trợ audit/log khi truy cập thất bại.

Checklist review

- Kiểm tra tất cả service cần bảo mật có import lifebalance-security không.
- Kiểm tra issuer-uri/jwk-set-uri trong Docker dùng keycloak:8080 nội bộ, không dùng localhost sai ngữ cảnh.
- Kiểm tra client-id/audience lifebalance-api khớp Keycloak client.
- Kiểm tra các endpoint public có thật sự nên public không.
- Kiểm tra controller luôn lấy current user từ security context, không tin ownerId client gửi lên.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Tại sao phải validate audience?	Vì token đúng issuer nhưng cấp cho client khác không nên được dùng để gọi API LifeBalance.
Stateless security có lợi gì?	Mỗi request tự mang token, service không cần session memory, phù hợp nhiều instance/microservice.

17. Module identity-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	identity-service
API/entry point	/auth, /api/auth, /users, /api/users, /roles, /permissions, /audit-logs, /administration-support
Dữ liệu sở hữu	users, roles, permissions, user_roles, role_permissions, audit_logs, activity_logs, support_tickets, announcements, configurations.

identity-service là service quản lý danh tính, quyền và quản trị. Đây là service nền nghiệp vụ vì hầu hết service khác đều cần biết user hiện tại là ai và có quyền gì. Identity không chỉ làm login; nó còn quản lý user, role, permission, audit log, support ticket, system announcement và system configuration.

Trong mô hình bảo mật hiện tại, Keycloak là nơi cấp token và quản lý danh tính bên ngoài. identity-service tích hợp với Keycloak admin client để đồng bộ hoặc quản trị user/role khi cần. Điều này tách hai lớp: Keycloak chịu trách nhiệm chuẩn OAuth2/OIDC, identity-service chịu trách nhiệm nghiệp vụ quản trị của ứng dụng LifeBalance.

Data model của identity có quan hệ nhiều-nhiều giữa User và Role thông qua UserRole, nhiều-nhiều giữa Role và Permission thông qua RolePermission. Đây là mô hình RBAC cơ bản. Role gom nhiều permission, user có

nhiều role. Khi cần kiểm soát quyền, hệ thống có thể kiểm tra role/permission thay vì chỉ dựa vào một cờ admin đơn giản.

AuditLog và ActivityLog dùng để ghi nhận hành động quan trọng. Khi bảo vệ, nên nhấn mạnh audit không phải chức năng phụ vô nghĩa; với hệ thống có tài chính, quyền và dữ liệu cá nhân, audit giúp truy vết ai đã làm gì, lúc nào, trên tài nguyên nào. SupportTicket và SupportTicketHistory phục vụ quản trị hỗ trợ người dùng. SystemAnnouncement và SystemConfiguration phục vụ thông báo/cấu hình hệ thống.

Review identity-service nên rất cẩn thận vì lỗi ở đây có thể ảnh hưởng toàn hệ thống. Nếu role sync sai, user có thể mất quyền hoặc có quyền quá mức. Nếu endpoint quản trị thiếu authorization, user thường có thể thao tác dữ liệu quản trị. Nếu audit không ghi đủ, khó truy vết sau sự cố.

Checklist review

- Kiểm tra endpoint quản trị yêu cầu role/permission phù hợp.
- Kiểm tra user update không cho tự nâng quyền trái phép.
- Kiểm tra mapping role/permission giữa database và Keycloak có nhất quán không.
- Kiểm tra audit log được ghi ở hành động nhạy cảm.
- Không trả thông tin nhạy cảm như password/hash/secret trong response DTO.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Identity-service khác Keycloak ở đâu?	Keycloak là identity provider/OIDC server; identity-service là service nghiệp vụ quản trị user/role/permission/audit trong hệ thống.
Vì sao cần role và permission?	Role giúp gán nhóm quyền nhanh; permission giúp kiểm soát hành động chi tiết hơn.

18. Module task-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	task-service
API/entry point	/api/tasks, /api/categories, /api/tags, /tags, /api/tasks/{taskId}/tags, /api/tasks/reminders, /api/tasks/recurring-rules, /api/tasks/timeline
Dữ liệu sở hữu	tasks, categories, tags, task_tags, task_reminders, task_recurring_rules, task_change_histories, timeline_placements.

task-service là service quản lý công việc. Đây là một trong các service trung tâm vì nhiều domain khác dựa vào task: timeline cần task để lập lịch, resource-capital cần task làm target phân bổ vốn, analytics cần task/actual record để đánh giá tiến độ, notification cần task để nhắc nhở.

Task domain không chỉ có bảng tasks. Nó còn có Category để nhóm task, Tag để gắn nhãn, TaskTag cho quan hệ nhiều-nhiều, TaskReminder để nhắc việc, TaskRecurringRule để mô tả task lặp lại, TaskChangeHistory để lưu lịch sử thay đổi. Cấu trúc này cho thấy service không chỉ CRUD đơn giản mà quản lý vòng đời task.

Controller của task-service nhận request từ gateway. Service layer kiểm tra owner, category/tag thuộc user hiện tại, trạng thái task hợp lệ, thời gian nhắc hợp lệ, recurring rule hợp lệ. Repository truy vấn task theo

ownerId để tránh user A thao tác task user B. Khi task thay đổi, service ghi lịch sử để phục vụ audit và analytics.

Task-service có integration client gọi sang timeline-service, notification-service và analytics-service. Ví dụ khi tạo/cập nhật task, service có thể đồng bộ snapshot sang timeline qua `/api/timeline/tasks`, tạo notification qua `/api/notifications`, hoặc record actual seed cho analytics qua `/api/analytics/actual-records`. Các call này cần bearer token và có thể có internal secret header.

Điểm cần hiểu sâu là task-service là nguồn sự thật của task, còn timeline-service chỉ giữ snapshot/placement phục vụ lịch. Nếu timeline cần hiển thị task, nó không nhất thiết sở hữu toàn bộ task gốc. Khi task không còn visible/eligible, timeline có thể đóng active placements. Đây là cách giảm coupling giữa hai service.

Checklist review

- Kiểm tra mọi query task quan trọng có ownerId/currentUser.
- Kiểm tra tạo/cập nhật task có validate category/tag/reminder/recurring rule.
- Kiểm tra lịch sử thay đổi được ghi đủ cho các field quan trọng.
- Kiểm tra integration failure không làm mất dữ liệu chính hoặc có chiến lược retry/outbox nếu cần.
- Kiểm tra endpoint `/api/tasks/timeline` và route gateway khớp nếu frontend dùng.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Task-service có phải chỉ CRUD không?	Không. Nó quản lý vòng đời task, category/tag, reminder, recurring rule, history và đồng bộ sang timeline/notification/analytics.
Nếu notification-service lỗi khi tạo task thì sao?	Task chính vẫn có thể được lưu; notification là side effect. Bản production nên dùng outbox/retry để tránh mất thông báo.

19. Module timeline-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	timeline-service
API/entry point	<code>/api/timeline</code> , <code>/api/timeline/placements</code> , <code>/api/timeline/day</code> , <code>/api/timeline/week</code> , <code>/api/timeline/availability</code> , <code>/api/timeline/tasks</code> , <code>/api/timeline/history</code>
Dữ liệu sở hữu	timeline_tasks, timeline_placements, timeline_histories.

timeline-service quản lý cách task được đặt vào lịch. Nếu task-service trả lời câu hỏi “việc gì cần làm”, timeline-service trả lời câu hỏi “việc đó được đặt vào khoảng thời gian nào, có xung đột không, lịch ngày/tuần trông ra sao”. Đây là lý do task và timeline được tách thành hai bounded context.

TimelineTask là snapshot hoặc bản đại diện của task trong timeline-service. TimelinePlacement là thông tin task được đặt vào khoảng thời gian cụ thể. TimelineHistory lưu lịch sử thay đổi placement. Cấu trúc này giúp timeline-service xử lý lịch mà không phải sở hữu toàn bộ bảng tasks của task-service.

Endpoint `/api/timeline/tasks` có vai trò nội bộ quan trọng. TimelineTaskController yêu cầu header X-Lifebalance-Internal-Secret và current user. requireTrustedInternalRequest so sánh secret bằng MessageDigest.isEqual để tránh so sánh chuỗi thông thường có rủi ro timing. Nếu internalSecret rỗng hoặc submitted secret rỗng hoặc không khớp, controller ném AccessDeniedException.

Khi task-service gọi upsert task sang timeline-service, timeline-service kiểm tra trusted internal request rồi gọi TimelineTaskService.upsertTask với ownerId hiện tại. Service có thể tìm task snapshot hiện có theo taskId + ownerId, cập nhật hoặc tạo mới, ghi history và đóng placement đang active nếu task không còn visible. Đây là logic đồng bộ dữ liệu liên service.

Timeline còn cung cấp API xem lịch ngày, tuần, availability và placement. Review timeline nên tập trung vào xung đột thời gian, owner isolation, timezone, trạng thái placement và cách xử lý khi snapshot task bị lỗi thời.

Checklist review

- Kiểm tra internal secret trong môi trường chạy không để rỗng nếu cần sync task -> timeline.
- Kiểm tra placement không cho chồng lịch sai nghiệp vụ.
- Kiểm tra timezone/ngày/tuần xử lý nhất quán.
- Kiểm tra ownerId trong mọi thao tác timeline task/placement.
- Kiểm tra khi task bị ẩn/xóa/complete thì placement active được xử lý đúng.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Vì sao timeline-service có timeline_tasks riêng?	Đó là snapshot phục vụ lập lịch, giúp timeline không phụ thuộc trực tiếp database task-service.
Internal secret để làm gì?	Để phân biệt request nội bộ service-to-service với request client thông thường, tăng lớp bảo vệ cho endpoint sync.

20. Module resource-capital-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	resource-capital-service
API/entry point	/api/v1/capital, /api/v1/capital-cycles, /api/v1/capital-adjustments, /api/v1/capital-allocations
Dữ liệu sở hữu	capital_cycles, time_capitals, money_capitals, capital_adjustments, capital_allocations, capital_histories, capital_reallocations, capital_releases.

resource-capital-service là service phức tạp về nghiệp vụ. Nó quản lý khái niệm vốn/capital của người dùng, gồm vốn thời gian và vốn tiền, chu kỳ vốn, điều chỉnh vốn, phân bổ vốn cho mục tiêu, giải phóng vốn và tái phân bổ. Đây không phải finance thuần túy, vì capital ở đây có thể là thời gian, tiền hoặc nguồn lực dùng để cân bằng cuộc sống.

CapitalCycle có thể xem là khung thời gian hoặc vòng đời quản lý vốn. Trong một cycle, người dùng có TimeCapital và MoneyCapital. CapitalAdjustment ghi nhận điều chỉnh tăng/giảm. CapitalAllocation ghi nhận phân bổ một phần vốn vào target như task. CapitalRelease giải phóng vốn đã phân bổ. CapitalReallocation chuyển vốn từ allocation này sang allocation khác. CapitalHistory lưu dấu vết thay đổi.

Service layer của module này có nhiều repository và validator. CapitalAllocationServiceImpl phụ thuộc AllocationService, CapitalCycleRepository, CapitalAllocationRepository, CapitalReallocationRepository, CapitalReleaseRepository, TimeCapitalRepository, MoneyCapitalRepository, CapitalHistoryRepository, DefaultAllocationValidator và CapitalIntegrationPublisher. Điều này cho thấy use case allocation không đơn giản: nó phải kiểm tra cycle, số dư vốn, target, lịch sử và side effect.

BasicAllocationTargetValidator gọi task-service để xác thực target task. Nó dùng base URL mặc định http://task-service:8080, truyền bearer token hiện tại và gọi GET /api/tasks/{id}. Nếu task-service trả 404/403/401 thì target không hợp lệ. Nếu lỗi mạng RestClientException thì ném AllocationTargetUnavailableException. Đây là ví dụ rõ ràng về service ownership: resource-capital không query trực tiếp database task.

Điểm cần bảo vệ là transaction cục bộ. Khi phân bổ vốn, service cần đảm bảo allocation, balance/history được cập nhật nhất quán trong database resource-capital. Nhưng xác thực target task là call sang service khác, có thể lỗi vì mạng. Vì vậy cần phân biệt lỗi nghiệp vụ “target không thuộc user” với lỗi hạ tầng “task-service đang unavailable”.

Checklist review

- Kiểm tra allocation không vượt số dư time/money capital.
- Kiểm tra target validation qua task-service có truyền token và xử lý 401/403/404 đúng.
- Kiểm tra transaction bao gồm allocation + history + balance update.
- Kiểm tra release/reallocation không tạo số âm hoặc double spend.
- Kiểm tra route gateway dùng capital số ít, tránh nhầm /capitals.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Resource-capital khác finance ở đâu?	Finance quản lý tài khoản/giao dịch tiền; resource-capital quản lý nguồn lực tổng thể gồm thời gian và tiền, đặc biệt là phân bổ vào mục tiêu/task.
Vì sao validator gọi task-service?	Vì task-service sở hữu task. Resource-capital chỉ cần xác minh target hợp lệ, không nên đọc database task trực tiếp.

21. Module finance-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	finance-service
API/entry point	/api/finance/accounts, /api/finance/categories, /api/budgets, /api/transactions, /api/finance/reports, /api/finance/recurring-transactions, /api/finance/history
Dữ liệu sở hữu	finance_accounts, finance_categories, finance_budgets, financial_transactions, recurring_transaction_rules, finance_histories.

finance-service quản lý tài chính cá nhân trong LifeBalance. Domain chính gồm tài khoản tài chính, category tài chính, ngân sách, giao dịch, giao dịch lặp lại, báo cáo và lịch sử. Nếu resource-capital trả lời “nguồn vốn được phân bổ vào đâu”, finance trả lời “đồng tiền thực tế vào/ra tài khoản và ngân sách thay đổi thế nào”.

FinanceAccount đại diện nguồn tiền như ví, ngân hàng, tài khoản tiết kiệm. FinanceCategory phân loại thu/chí. FinanceBudget đặt giới hạn hoặc kế hoạch chi tiêu. FinancialTransaction ghi nhận giao dịch. RecurringTransactionRule mô tả giao dịch lặp lại như tiền thuê nhà hằng tháng. FinanceHistory giúp truy vết thay đổi tài chính.

Khi viết finance-service, điều quan trọng là tính nhất quán số tiền. Service phải kiểm tra owner, account/category thuộc user, amount hợp lệ, currency nếu có, trạng thái void/cancel không bị xử lý hai lần.

Nếu transaction liên kết capitalCycleId hoặc capitalAllocationId, cần xác định rõ đây là reference sang resource-capital chứ không phải FK xuyên database.

Báo cáo tài chính cần aggregate dữ liệu theo thời gian/category/account. Đây là phần có rủi ro performance khi dữ liệu lớn. Review nên kiểm tra pagination/filter, index theo ownerId/date/category/account, và tránh trả toàn bộ transaction nếu client chỉ cần tổng hợp.

Với dữ liệu tài chính, audit/history quan trọng hơn module thông thường. Khi giao dịch bị sửa hoặc void, nên lưu lịch sử để người dùng hiểu vì sao số dư thay đổi. Nếu sau này production, có thể cần immutable ledger hoặc double-entry accounting tùy độ nghiêm túc của domain.

Checklist review

- Kiểm tra transaction amount không âm sai ngữ cảnh và không overflow.
- Kiểm tra account/category/budget đều thuộc owner hiện tại.
- Kiểm tra sửa/xóa/void transaction có ghi FinanceHistory.
- Kiểm tra báo cáo có filter theo owner/date và có index phù hợp.
- Nếu liên kết capital allocation, cần quy định consistency khi allocation thay đổi.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Finance-service có trùng resource-capital không?	Không. Finance ghi dòng tiền/giao dịch; resource-capital quản lý kế hoạch/phân bổ nguồn lực.
Vì sao cần finance history?	Vì dữ liệu tiền cần truy vết thay đổi để giải thích số dư và phục vụ audit.

22. Module notification-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	notification-service
API/entry point	/api/notifications, /api/notifications/templates, /api/notifications/preferences, /api/notifications/delivery, /api/notifications/history
Dữ liệu sở hữu	notifications, notification_templates, notification_preferences, notification_delivery_attempts, notification_histories.

notification-service quản lý thông báo. Trong LifeBalance, notification có thể đến từ task reminder, thay đổi timeline, cảnh báo budget, insight AI hoặc thông báo hệ thống. Tách notification thành service riêng giúp các service nghiệp vụ không phải tự xử lý template, preference, retry và lịch sử gửi.

NotificationRecord là bản ghi thông báo. NotificationTemplate định nghĩa nội dung mẫu.

NotificationPreference lưu lựa chọn người dùng như bật/tắt loại thông báo, kênh gửi, thời gian yên lặng.

NotificationDeliveryAttempt ghi từng lần thử gửi. NotificationHistory lưu lịch sử thay đổi/trạng thái.

Trong hệ thống hiện tại, một số service có thể gọi notification-service qua REST để tạo thông báo. Với cách này, service gọi phải quyết định nếu notification-service lỗi thì nghiệp vụ chính có rollback không. Thường thông báo là side effect, nên không nên làm hỏng use case chính như tạo task hoặc phân bổ vốn. Tuy nhiên cần cơ chế retry/outbox để thông báo không mất vĩnh viễn.

Review notification-service nên chú ý idempotency. Nếu task-service retry tạo notification, có thể sinh trùng thông báo. Cần có deduplication key hoặc logic kiểm tra để tránh gửi nhiều lần cùng một thông báo. Delivery attempt cũng cần phân biệt lỗi tạm thời và lỗi vĩnh viễn.

Preference là điểm bảo vệ về trải nghiệm và quyền riêng tư. Người dùng có thể không muốn nhận một số loại thông báo hoặc không muốn nhận trong quiet hours. Notification-service là nơi đúng để gom logic này thay vì mỗi service tự kiểm tra một kiểu.

Checklist review

- Kiểm tra preference được áp dụng trước khi gửi thông báo.
- Kiểm tra delivery attempt có retry/backoff hoặc ít nhất lưu lỗi.
- Kiểm tra tạo notification có idempotency/dedup nếu service gọi retry.
- Kiểm tra không gửi lộ dữ liệu nhạy cảm trong nội dung thông báo.
- Kiểm tra endpoint nội bộ nếu có yêu cầu secret/token phù hợp.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Vì sao tách notification-service?	Để gom template, preference, delivery, retry/history vào một nơi thay vì lặp ở từng service.
Thông báo lỗi có làm fail tạo task không?	Thông thường không nên; notification là side effect. Nhưng cần retry/outbox để tránh mất thông báo.

23. Module analytics-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	analytics-service
API/entry point	/api/analytics/dashboard, /api/analytics/reports, /api/analytics/tracking-evaluation, /api/analytics/evaluations, /api/analytics/actual-records, /api/analytics/history
Dữ liệu sở hữu	actual_records, analytics_reports, evaluation_results, analytics_histories.

analytics-service chịu trách nhiệm phân tích dữ liệu, dashboard, report và đánh giá tiến độ. Nó không nên trở thành nơi ghi dữ liệu gốc của mọi domain. Thay vào đó, nó lưu actual records, report, evaluation result và history phục vụ đọc/tổng hợp. Nguồn sự thật của task vẫn là task-service, nguồn sự thật tài chính vẫn là finance-service.

ActualRecord là dữ liệu thực tế dùng để đánh giá, ví dụ task hoàn thành, thời lượng thực hiện, giá trị đo lường. EvaluationResult là kết quả đánh giá từ dữ liệu thực tế so với mục tiêu/kế hoạch. AnalyticsReport là báo cáo được tạo theo loại/dimension/khoảng thời gian. AnalyticsHistory lưu lịch sử phân tích hoặc export.

Service này thường có dạng read model/analysis model. Nó có thể nhận dữ liệu seed từ task-service hoặc tổng hợp từ các service khác. Vì dữ liệu phân tích thường không cần consistency tức thời tuyệt đối, kiến trúc event-driven sẽ phù hợp nếu nâng cấp. Hiện tại REST vẫn đủ cho đồ án nếu dữ liệu và yêu cầu không quá lớn.

Review analytics cần chú ý performance. Dashboard/report dễ bị query nặng nếu aggregate nhiều bản ghi mà không có index/filter. Cần luôn filter theo ownerId và khoảng thời gian. Cũng cần phân biệt dữ liệu real-time và dữ liệu batch: nếu report được generate và lưu trạng thái, client không nên chờ request quá lâu.

Khi bảo vệ, có thể nói analytics-service tách riêng vì phân tích và báo cáo có pattern truy vấn khác CRUD nghiệp vụ. Nó có thể tối ưu database/index/cache riêng mà không ảnh hưởng task/finance.

Checklist review

- Kiểm tra mọi report/dashboard filter theo ownerId.
- Kiểm tra query aggregate có giới hạn thời gian/pagination.
- Kiểm tra report export không block request quá lâu nếu dữ liệu lớn.
- Kiểm tra dữ liệu actual record không bị ghi trùng khi service nguồn retry.
- Cân nhắc cache/materialized view nếu dashboard chậm.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Analytics có phải database chung không?	Không. Nó là read/analysis model, không thay thế database gốc của từng service.
Vì sao analytics nên tách?	Vì truy vấn báo cáo/tổng hợp có nhu cầu tối ưu khác nghiệp vụ CRUD, tách ra giảm ảnh hưởng service gốc.

24. Module ai-service

Thông tin	Nội dung
Nhóm	Service nghiệp vụ
Đường dẫn chính	ai-service
API/entry point	/api/ai/recommendations, /api/ai/insights, /api/ai/conversations, /api/ai/history
Dữ liệu sở hữu	ai_recommendations, ai_insights, ai_conversations, ai_messages, ai_histories.

ai-service quản lý các chức năng thông minh như recommendation, insight, conversation và history. Tách AI thành service riêng là hợp lý vì logic AI thường thay đổi nhanh, phụ thuộc provider/model, có timeout/chi phí/giới hạn rate riêng và không nên làm phức tạp các service nghiệp vụ lõi.

AiRecommendation lưu gợi ý cho người dùng. AiInsight lưu nhận định hoặc phân tích từ dữ liệu LifeBalance. AiConversation và AiMessage lưu hội thoại. AiHistory giúp truy vết quá trình sinh gợi ý/insight, phục vụ debug hoặc audit. Các dữ liệu này cần gắn ownerId để không lẫn dữ liệu giữa người dùng.

AI-service không nên tự ý đọc database của các service khác. Nếu cần ngữ cảnh task/finance/timeline, nên lấy qua API hoặc qua dữ liệu đã được đồng bộ/analytics. Điều này giữ ranh giới service. Với AI, cần chú ý privacy: prompt gửi ra provider không nên chứa dữ liệu nhạy cảm quá mức nếu không cần.

Review AI-service cần chú ý fallback. Nếu provider AI lỗi, timeout hoặc trả kết quả không hợp lệ, service nên trả lỗi rõ ràng hoặc dùng fallback, không làm treo toàn bộ request. Cũng cần kiểm soát input/output để tránh prompt injection hoặc nội dung không phù hợp nếu sản phẩm triển khai thật.

Khi bảo vệ, nên nói AI-service là module mở rộng giá trị sản phẩm, không phải nền tảng bắt buộc để task/finance chạy. Việc tách riêng giúp hệ thống lõi vẫn ổn định ngay cả khi AI provider thay đổi hoặc tạm thời không hoạt động.

Checklist review

- Kiểm tra ownerId trên conversation/message/recommendation.

- Kiểm tra timeout/fallback khi provider AI lỗi.
- Kiểm tra không log hoặc gửi dữ liệu nhạy cảm không cần thiết.
- Kiểm tra lịch sử AI đủ để debug nhưng không vi phạm quyền riêng tư.
- Nếu recommendation ảnh hưởng quyết định tài chính/thời gian, cần ghi rõ đây là gợi ý, không phải quyết định bắt buộc.

Câu hỏi bảo vệ nhanh

Câu hỏi	Cách trả lời
Vì sao AI-service tách riêng?	Vì AI có phụ thuộc provider, timeout, chi phí và tốc độ thay đổi khác service lõi.
AI lỗi có làm hệ thống chính lỗi không?	Không nên. Task/finance/timeline vẫn phải chạy; AI chỉ là module tăng giá trị.

25. Review Lỗi, Mức Ảnh Hưởng Và Hướng Sửa

Phần này trả lời trực tiếp câu hỏi “có cần sửa không” và “những lỗi này ảnh hưởng lớn không”. Cần phân biệt lỗi làm sập hệ thống, lỗi làm sai API contract, lỗi làm test không đáng tin, lỗi vận hành, và điểm cải tiến production.

Vấn đề	Mức ảnh hưởng	Giải thích	Trạng thái/hướng xử lý
Discovery hostname trong Docker/prod	Vừa đến lớn khi deploy Docker	Nếu Eureka quảng bá hostname sai, gateway/service có thể không gọi được instance.	Đã cấu hình EUREKA_INSTANCE_HOSTNAME default discovery-server và Compose truyền biến tương ứng.
Gateway route /api/v1/capital vs /api/v1/capitals	Vừa nếu frontend/test dùng sai path	Route hiện tại dùng capital số ít. Test/client dùng capitals có thể 404.	Cần thống nhất API contract: đổi route hoặc sửa client/test về đúng path.
/actuator/prometheus trong test trả 401 nhưng test kỳ vọng 200	Vừa với monitoring/test	Nếu Prometheus scrape không token mà endpoint bị khóa, monitoring fail. Nếu test sai profile, test không phản ánh runtime.	Cần thống nhất security rule và profile test; lifebalance-security có permit prometheus, nhưng gateway test context cần kiểm tra lại.
Internal secret mặc định có thể rỗng	Vừa với service-to-service sync	TimelineTaskController từ chối request nếu internalSecret rỗng hoặc header rỗng. Nếu môi trường không cấu hình secret, task -> timeline sync có thể fail.	Cần set LIFEBALANCE_INTERNAL_SERVICE_SECRET trong .env khi dùng integration; cần nhắc fail-fast nếu secret bắt buộc.
REST side effect chưa có outbox/retry mạnh	Vừa, tăng theo quy mô production	Task tạo thành công nhưng notification/timeline/analytics có thể chưa đồng bộ khi service đích lỗi.	Phù hợp đồ án; production nên dùng outbox + message broker hoặc retry/circuit breaker.
Các service dùng cùng PostgreSQL instance	Thấp đến vừa	Ranh giới database logical tốt cho đồ án, nhưng hạ tầng vật lý chung vẫn là single point of failure.	Nếu production, tách instance/backup/permission chặt hơn.
Chưa thấy tracing/correlation id	Vừa khi debug production	Microservice khó debug nếu không theo dấu request qua	Nên thêm correlation id, centralized logging, distributed tracing.

Vấn đề	Mức ảnh hưởng	Giải thích	Trạng thái/hướng xử lý
toàn hệ thống		gateway và service.	
Test coverage không đều	Vừa	Một số service domain phức tạp nhưng test ít hơn, dễ bỏ sót edge case.	Ưu tiên bổ sung test cho timeline, finance, notification, ai, gateway route/security.

Những vấn đề trên không có cùng mức nghiêm trọng. Discovery hostname nếu sai trong Docker có thể làm service discovery hỏng nên đã cần sửa. Gateway route mismatch có thể làm API cụ thể bị lỗi nhưng không làm toàn backend sập. Prometheus 401 ảnh hưởng monitoring/test hơn là nghiệp vụ người dùng. REST side effect chưa có outbox là limitation kiến trúc; với đồ án có thể chấp nhận nếu giải thích rõ, nhưng với production cần nâng cấp.

Khi bảo vệ, không nên nói “hệ thống không có lỗi”. Câu trả lời tốt hơn là: “Em đã review và phân loại rủi ro. Những lỗi cấu hình discovery đã được sửa. Các điểm còn lại chủ yếu là consistency/observability/test coverage; em biết hướng nâng cấp bằng outbox, message broker, circuit breaker, tracing và bổ sung test.” Cách trả lời này thể hiện tư duy kỹ thuật trưởng thành.

26. Bộ Câu Hỏi Phản Biện Và Cách Trả Lời Sâu

Câu hỏi	Cách trả lời chi tiết
Backend của em dùng kiến trúc gì?	Backend dùng hướng microservice trên Spring Boot. Về build-time, dự án là Maven multi-module với 12 module. Về runtime, các service chạy độc lập qua Docker Compose. Gateway là cửa vào API, Eureka làm service discovery, Keycloak cấp JWT, PostgreSQL lưu dữ liệu với database logical riêng cho từng service. Em gọi là microservice-oriented vì có tách bounded context, service registry, gateway và data ownership theo service.
Vì sao không dùng monolith cho đơn giản?	Monolith đơn giản lúc đầu nhưng LifeBalance có nhiều domain khác nhau: identity, task, timeline, resource capital, finance, notification, analytics, AI. Tách service giúp mỗi domain có code, database và lifecycle rõ hơn. Đổi lại hệ thống phức tạp hơn về vận hành và consistency, nên em dùng gateway, discovery, security common, Docker healthcheck và xác định các điểm cần nâng cấp như outbox/tracing.
Gateway làm gì?	Gateway nhận request từ frontend, kiểm tra route Path, xác thực JWT qua security chung và định tuyến đến service bằng lb://SERVICE-NAME. Nó không xử lý nghiệp vụ. Ví dụ /api/tasks/** đi đến TASK-SERVICE, /api/finance/** đi đến FINANCE-SERVICE. Gateway giúp frontend chỉ cần gọi một endpoint thay vì biết địa chỉ từng service.
Discovery-server làm gì?	Discovery-server là Eureka registry. Service khi khởi động đăng ký tên và địa chỉ với Eureka. Gateway dùng registry để tìm instance thay vì hard-code host. Nếu scale nhiều instance task-service, gateway có thể load balance qua service name. Discovery không xử lý request nghiệp vụ.
JWT được kiểm tra ở đâu?	JWT được kiểm tra bởi lifebalance-security trong Gateway và các service import module này. JwtDecoder kiểm tra chữ ký/JWK, issuer, thời hạn và audience client-id. Sau đó KeycloakUserMappingFilter map thông tin token thành principal để controller/service lấy ownerId hiện tại.
Nếu user A cố đọc dữ liệu user B thì sao?	Token hợp lệ chỉ chứng minh user A là ai. Service vẫn phải kiểm tra ownerId ở business layer/repository. Query nên theo id + ownerId, không chỉ findById. Nếu không thuộc user hiện tại thì trả 403 hoặc 404 tùy chính sách. Đây là lỗi

Câu hỏi	Cách trả lời chi tiết
	authorization theo dữ liệu.
Vì sao task và timeline tách riêng?	Task quản lý bản chất công việc: title, status, category, tag, reminder, recurring rule. Timeline quản lý việc đặt task vào lịch: placement, ngày/tuần, availability, history. Timeline có snapshot task riêng để lập lịch, nhưng task-service vẫn là nguồn sự thật của task.
Nếu tạo task thành công nhưng đồng bộ timeline thất bại thì sao?	Tạo task là transaction cục bộ của task-service. Đồng bộ timeline là side effect liên service qua REST. Hiện tại có thể log warning và task vẫn tồn tại. Với production nên dùng outbox/message broker để đảm bảo event không mất và có retry.
Database có thật sự tách không?	Mỗi service có database logical riêng và datasource riêng, ví dụ IDENTITY_DB, TASK_DB, FINANCE_DB. Chúng cùng nằm trong một PostgreSQL container trong Docker Compose để dễ triển khai đồ án. Về nguyên tắc ownership, service không nên đọc trực tiếp database của service khác.
Controller-Service-Repository khác nhau thế nào?	Controller là lớp HTTP adapter, nhận request và gọi service. Service chứa nghiệp vụ, transaction, ownership, validation nghiệp vụ, gọi integration. Repository truy cập database qua JPA. Tách vậy giúp test và bảo trì tốt hơn.
Module nào quan trọng nhất?	Tùy góc nhìn. Về hạ tầng, gateway và discovery rất quan trọng vì request và routing phụ thuộc vào chúng. Về bảo mật, identity và lifebalance-security rất quan trọng. Về nghiệp vụ cốt lõi, task, timeline, resource-capital và finance là các module chính của sản phẩm.
Những lỗi hiện tại có nghiêm trọng không?	Có lỗi cấu hình/discovery cần sửa vì ảnh hưởng deploy Docker; phần đó đã cấu hình hostname discovery-server. Các lỗi gateway route/test ảnh hưởng API contract và monitoring nhưng không làm toàn hệ thống sập. Các điểm như outbox/tracing/test coverage là limitation cần nâng cấp khi production.
Tại sao cần lifebalance-common?	Để response/error/pagination/status thống nhất. Nếu mỗi service tự định nghĩa lỗi, frontend khó xử lý và review API khó. Common giữ các class trung lập như ApiResponse, ApiError, AppException, PageableLimits.
Tại sao cần lifebalance-security thay vì copy config?	Security là cross-cutting concern. Nếu copy từng service, dễ lệch rule và khó sửa. Đưa vào module auto-configuration giúp tất cả service dùng chung logic JWT, principal mapping và handler 401/403.
Hệ thống có thể mở rộng thế nào?	Có thể scale từng service độc lập, ví dụ task-service nhiều instance nếu request task nhiều. Gateway dùng Eureka để tìm instance. Có thể thêm RabbitMQ/outbox cho event, Redis cache cho dashboard, Prometheus/Grafana cho monitoring, tracing để debug request xuyên service.

27. Cách Học Báo Cáo Này Để Bảo Vệ

Không nên học thuộc toàn bộ báo cáo theo từng chữ. Cách hiệu quả là học theo lớp. Đầu tiên nắm kiến trúc tổng thể và số module. Sau đó học luồng request tạo task vì luồng này đi qua gateway, security, service, repository, database và integration. Tiếp theo học từng module theo vai trò, dữ liệu sở hữu và API chính. Cuối cùng luyện câu hỏi phản biện.

- Học câu trả lời 30 giây: Backend có 12 module, theo hướng microservice, Gateway + Eureka + Keycloak + PostgreSQL.
- Học sơ đồ request: Client -> Gateway -> Eureka -> Service -> Security -> Controller -> Service -> Repository -> DB -> Response.

3. Học 4 module nền: discovery-server, gateway, lifebalance-common, lifebalance-security.
4. Học 8 service nghiệp vụ theo cặp: identity/security, task/timeline, resource-capital/finance, notification/analytics/ai.
5. Với mỗi service, trả lời 5 ý: làm gì, API chính, data chính, code layer, rủi ro review.
6. Học kỹ khác biệt task vs timeline và finance vs resource-capital vì đây là câu hỏi dễ bị bắt bẻ.
7. Học security: authentication khác authorization, JWT kiểm tra issuer/audience/expired, ownerId chống đọc dữ liệu người khác.
8. Học database: database logical riêng trong cùng PostgreSQL container, không query xuyên service.
9. Học lỗi đã review: discovery hostname, gateway route/test, internal secret, outbox/tracing/test coverage.
10. Luyện trả lời bằng lập luận: nói ưu điểm, nhược điểm, hướng cải tiến; không nói tuyệt đối rằng hệ thống hoàn hảo.

28. Phụ Lục Kỹ Thuật A: Bảng Route Gateway Chi Tiết

Bảng này dùng khi hội đồng hỏi gateway định tuyến cụ thể như thế nào. Điểm quan trọng là gateway không biết nghiệp vụ bên trong service, nhưng nó biết URL prefix nào thuộc service nào. Khi request đến, gateway so khớp Path predicate, chọn route và dùng lb:// để tra service instance qua Eureka.

Route id	Path predicate	URI đích	Ý nghĩa
identity-service	/api/identity/**, /api/v1/identity/**, /api/auth/**, /auth/**, /api/users/**, /users/**, /api/roles/**, /roles/**, /api/permissions/**, /permissions/**, /api/administration-support/**, /administration-support/**, /api/audit-logs/**, /audit-logs/**	lb://IDENTITY-SERVICE	Gom toàn bộ API xác thực, user, role, permission, audit và quản trị.
task-service	/api/tasks/**, /api/categories/**, /api/tags/**, /tags/**	lb://TASK-SERVICE	Định tuyến nghiệp vụ task, category, tag và các thao tác liên quan.
resource-capital-service	/api/v1/capital/**, /api/v1/capital-cycles/**, /api/v1/capital-adjustments/**, /api/v1/capital-allocations/**	lb://RESOURCE-CAPITAL-SERVICE	Định tuyến quản lý vốn, chu kỳ vốn, điều chỉnh và phân bổ vốn.
timeline-service	/api/timeline/**	lb://TIMELINE-SERVICE	Định tuyến toàn bộ timeline, placement, availability và task snapshot.
finance-service	/api/finance/**, /api/budgets/**, /api/transactions/**	lb://FINANCE-SERVICE	Định tuyến tài khoản tài chính, giao dịch, budget, report.
notification-service	/api/notifications/**	lb://NOTIFICATION-SERVICE	Định tuyến thông báo, template, preference, delivery, history.

Route id	Path predicate	URI đích	Ý nghĩa
ai-service	/api/ai/**	lb://AI-SERVICE	Định tuyến recommendation, insight, conversation, history AI.
analytics-service	/api/analytics/**	lb://ANALYTICS-SERVICE	Định tuyến dashboard, report, evaluation, actual record, analytics history.

Khi review route, hãy kiểm tra từ ngoài vào trong: frontend gọi path nào, gateway có Path predicate khớp không, service name trong lb:// có khớp tên đăng ký Eureka không, controller trong service có @RequestMapping tương ứng không. Một request sai có thể lỗi ở bất cứ chặng nào: không khớp route ở gateway, không tìm thấy service trong Eureka, hoặc vào service nhưng controller không có endpoint.

Ví dụ route resource-capital dùng /api/v1/capital/** dạng số ít. Nếu tài liệu frontend hoặc test dùng /api/v1/capitals/** dạng số nhiều, lỗi sẽ biểu hiện như 404. Đây là lỗi API contract, không phải lỗi database hay lỗi service discovery. Cách debug là nhìn log gateway, kiểm tra route match và kiểm tra controller mapping.

29. Phụ Lục Kỹ Thuật B: Quan Hệ Dữ Liệu Theo Module

Bảng dưới đây không thay thế ERD đầy đủ, nhưng đủ để giải thích quan hệ dữ liệu khi bảo vệ. Mục tiêu là nói được mỗi module có root entity nào, entity phụ nào và quan hệ nghiệp vụ là gì.

Module	Quan hệ dữ liệu quan trọng	Cách giải thích khi bảo vệ
identity-service	User nhiều-nhiều Role qua UserRole; Role nhiều-nhiều Permission qua RolePermission; SupportTicket liên quan requester/assignee; AuditLog/ActivityLog ghi hành động.	Identity dùng RBAC. User không chứa quyền trực tiếp mà đi qua role/permission để dễ quản trị.
task-service	Task nhiều-một Category; Task nhiều-nhiều Tag qua TaskTag; Task một-nhiều Reminder/RecurringRule/ChangeHistory; TimelinePlacement tham chiếu task trong domain task.	Task là aggregate trung tâm, category/tag giúp phân loại, reminder/recurring/history quản lý vòng đời.
timeline-service	TimelinePlacement nhiều-một TimelineTask; TimelineHistory tham chiếu placement/task; TimelineTask là snapshot từ task-service.	Timeline không sở hữu task gốc, chỉ giữ bản phục vụ lập lịch và placement.
resource-capital-service	CapitalCycle liên quan TimeCapital/MoneyCapital; Allocation/Adjustment/Release/Reallocation/History xoay quanh cycle và balance.	CapitalCycle là bối cảnh quản lý vốn; mọi phân bổ/điều chỉnh cần giữ balance không âm và có lịch sử.
finance-service	FinanceAccount, FinanceCategory, FinanceBudget liên quan FinancialTransaction; RecurringTransactionRule sinh giao dịch lặp; FinanceHistory ghi thay đổi.	Finance lưu dòng tiền thực tế và báo cáo, khác với kế hoạch phân bổ vốn của resource-capital.

Module	Quan hệ dữ liệu quan trọng	Cách giải thích khi bảo vệ
notification-service	NotificationRecord dùng Template/Preference; DeliveryAttempt ghi từng lần gửi; NotificationHistory ghi lifecycle.	Notification tách riêng để kiểm soát preference, retry, template và lịch sử gửi.
analytics-service	ActualRecord là dữ liệu thực tế; EvaluationResult là kết quả đánh giá; AnalyticsReport là báo cáo; AnalyticsHistory là lịch sử.	Analytics là read/analysis model, không thay thế nguồn dữ liệu gốc.
ai-service	AiConversation một-nhiều AiMessage; Recommendation/Insight/History gắn user/context.	AI lưu kết quả thông minh và hội thoại, nhưng phải kiểm soát privacy và owner.

Điểm cần nhấn mạnh là quan hệ nội bộ service khác quan hệ xuyên service. Quan hệ nội bộ có thể dùng JPA relationship hoặc foreign key vì cùng database ownership. Quan hệ xuyên service nên dùng id tham chiếu và API/event để xác minh. Ví dụ resource-capital lưu targetId của task và gọi task-service để xác thực, thay vì tạo foreign key sang database task.

30. Phụ Lục Kỹ Thuật C: Hợp Đồng Gọi Liên Service

Microservice không thể tránh giao tiếp liên service. Nhưng mỗi call liên service cần được coi là một hợp đồng: service gọi là ai, service nhận là ai, endpoint nào, cần token/secret gì, nếu lỗi thì nghiệp vụ chính có fail không. Bảng này là phần hội đồng thường đánh giá cao vì nó cho thấy bạn hiểu hệ thống chạy thật chứ không chỉ hiểu class diagram.

Service gọi	Service nhận	Endpoint/ý nghĩa	Bảo mật	Khi lỗi thì sao
task-service	timeline-service	POST /api/timeline/tasks để upsert snapshot task phục vụ lịch.	Bearer token + X-Lifebalance-Internal-Secret.	Thường log warning; task chính không nhất thiết rollback. Nên nâng cấp outbox/retry.
task-service	notification-service	POST /api/notifications để tạo thông báo nhắc việc/thay đổi.	Bearer token, có thể kèm internal secret tùy endpoint.	Notification là side effect; cần retry/idempotency nếu production.
task-service	analytics-service	POST /api/analytics/actual-records để seed dữ liệu đánh giá.	Bearer token.	Nếu analytics lỗi, task vẫn nên tồn tại; dữ liệu phân tích có thể đồng bộ lại sau.
resource-capital-service	task-service	GET /api/tasks/{id} để xác thực allocation target thuộc user.	Bearer token hiện tại.	401/403/404 nghĩa target invalid; lỗi mạng nghĩa target validation unavailable.
resource-capital-service	notification-service	Tạo thông báo khi phân bổ/điều chỉnh vốn.	Bearer token hoặc internal credential.	Không nên làm hỏng transaction vốn nếu chỉ là thông báo; cần retry nếu thông báo quan trọng.

Có ba chiến lược xử lý lỗi liên service. Chiến lược thứ nhất là fail-fast: service nhận lỗi thì use case chính thất bại. Phù hợp khi dữ liệu bắt buộc, ví dụ resource-capital không thể phân bổ vào task nếu không xác minh target. Chiến lược thứ hai là best-effort: lỗi side effect được log, use case chính vẫn thành công. Phù hợp với

notification. Chiến lược thứ ba là eventual consistency bằng outbox/event: use case chính commit cùng event, event được retry cho đến khi service đích xử lý. Đây là hướng nâng cấp tốt nhất cho production.

31. Phụ Lục Kỹ Thuật D: Playbook Review Code Backend

Khi được giao review một module, đừng bắt đầu bằng việc đọc từng file từ đầu đến cuối. Hãy đi theo luồng use case. Ví dụ review tạo task thì tìm controller endpoint POST /api/tasks, đọc request DTO, đọc service method, đọc repository query, đọc entity/migration, đọc exception handler và test. Cách review theo luồng giúp phát hiện lỗi thực tế nhanh hơn.

11. Xác định endpoint hoặc use case cần review: ví dụ tạo task, phân bổ capital, tạo transaction.
12. Kiểm tra route gateway có đưa request đến đúng service không.
13. Kiểm tra controller: HTTP method, path, @Valid, lấy currentUser, status code, response wrapper.
14. Kiểm tra request DTO: trường bắt buộc, giới hạn size, enum, nested validation.
15. Kiểm tra service: owner check, business rule, transaction, lịch sử, side effect.
16. Kiểm tra repository: query có ownerId không, pagination/filter, lock nếu cần chống race condition.
17. Kiểm tra entity/database: table, column, enum, index, migration, quan hệ, soft delete/history.
18. Kiểm tra error handling: lỗi validation/business/auth trả status và code hợp lý.
19. Kiểm tra security: endpoint public/private, role/permission, internal secret nếu endpoint nội bộ.
20. Kiểm tra test: có test success path, validation error, forbidden/unauthorized, edge case và integration failure không.

Một reviewer tốt không chỉ hỏi “code chạy không” mà hỏi “code sai ở biên thì sao”. Ví dụ: page size quá lớn thì sao; user A gửi categoryId của user B thì sao; service đích timeout thì sao; request retry hai lần thì có tạo trùng không; transaction rollback có để lại history mờ mờ không; token hết hạn trả 401 hay 500.

32. Phụ Lục Kỹ Thuật E: Tình Huống Lỗi Vận Hành Và Cách Giải Thích

Tình huống	Triệu chứng	Nguyên nhân hay gặp	Cách kiểm tra
Gateway gọi service bị lỗi	Client nhận 503/404 hoặc route không tới service.	Service chưa đăng ký Eureka, service name sai, route path sai.	Mở Eureka UI, xem service registered; kiểm tra gateway application.yaml.
Service không kết nối PostgreSQL	Service unhealthy, log datasource connection error.	Dùng localhost trong container, sai DB/user/password, postgres chưa healthy.	Kiểm tra SPRING_DATASOURCE_URL và docker compose ps/logs.
JWT bị từ chối	401 Unauthorized.	Issuer URI sai, JWK set URI sai, token hết hạn, audience không khớp.	Kiểm tra Keycloak realm/client, KEYCLOAK_ISSUER_URI_INTERNAL, client-id.
Có token nhưng vẫn bị 403	403 Forbidden.	Thiếu role/permission, owner check fail, internal secret sai.	Kiểm tra claim token, quyền user, header X-Lifebalance-Internal-Secret.
Prometheus không scrape được	Monitoring không có metrics gateway/service.	Endpoint prometheus bị security chặn hoặc network profile chưa bật.	Gọi /actuator/prometheus, kiểm tra security rule và profile monitoring.
Task tạo nhưng timeline không thấy	Task tồn tại nhưng lịch không có	task -> timeline sync lỗi, internal secret rỗng/sai,	Kiểm tra log task-service/timeline-service, secret env, endpoint

Tình huống	Triệu chứng	Nguyên nhân hay gặp	Cách kiểm tra
	snapshot/placement.	timeline down.	/api/timeline/tasks.
Notification bị trùng	Người dùng nhận nhiều thông báo giống nhau.	Retry không có idempotency key/dedup.	Kiểm tra delivery attempt, notification record và logic create.
Dashboard chậm	API analytics timeout/chậm.	Aggregate nhiều dữ liệu, thiếu index/cache, query không giới hạn thời gian.	Kiểm tra query, index owner/date, pagination và cache.

Cách trả lời tốt khi gặp tình huống lỗi là đi theo từng lớp: request có đến gateway không, gateway có match route không, Eureka có instance không, service có healthy không, security có chặn không, controller có mapping không, service có ném ApplicationException không, database có lỗi không. Cách debug theo lớp thể hiện bạn hiểu kiến trúc vận hành.

33. Phụ Lục Kỹ Thuật F: Lộ Trình Nâng Cấp Production

Hạng mục	Vì sao cần	Cách nâng cấp đề xuất
Outbox/event-driven integration	Tránh mất side effect khi service đích down.	Mỗi service lưu event trong DB cùng transaction, worker publish RabbitMQ/Kafka, consumer idempotent.
Circuit breaker/timeout/retry	Tránh một service chậm kéo sập service gọi.	Dùng Resilience4j, timeout ngắn, retry có giới hạn, fallback rõ.
Distributed tracing	Debug request xuyên gateway và nhiều service khó nếu chỉ có log rời.	Thêm correlation id, OpenTelemetry, trace id trong log.
Centralized logging	Log nằm rải rác từng container khó phân tích.	Đưa log về Loki/ELK/OpenSearch, gắn service name/trace id/user id ẩn danh.
Secret management	.env phù hợp local nhưng không đủ production.	Dùng Docker secrets/Kubernetes secrets/Vault, rotate secret định kỳ.
Database hardening	Cùng PostgreSQL container là single point of failure.	Backup/restore, user permission riêng, replication, monitoring, tách instance nếu cần.
API contract test	Gateway route, frontend và controller dễ lệch.	Dùng OpenAPI, contract test, CI kiểm tra route/controller.
Security hardening	Dữ liệu user/tài chính/AI có rủi ro quyền riêng tư.	Kiểm tra owner mọi query, audit action nhạy cảm, mask log, rate limit, CORS chặt.

34. Nguồn Code/Cấu Hình Đã Dựa Vào

Danh sách dưới đây giúp khi bị hỏi “em căn cứ vào đâu” có thể chỉ ra file thật trong project. Không cần đọc từng file khi thuyết trình, nhưng nên biết các file này tồn tại và chứa vai trò gì.

Nhóm	File/thư mục	Nội dung dùng trong báo cáo
Parent build	lifebalance-backend/pom.xml	Khai báo Spring Boot 3.5.16, Java 21, Spring Cloud 2025.0.3 và 12 module Maven.
Gateway	gateway/src/main/resources/application.yaml	Route Path và lb://SERVICE-NAME cho 8 service nghiệp vụ.

Nhóm	File/thư mục	Nội dung dùng trong báo cáo
Gateway	gateway/pom.xml	Dependency Spring Cloud Gateway WebMVC, Eureka Client, Actuator, OAuth2 Resource Server, Security.
Discovery	discovery-server/src/main/resources/application.yaml	Eureka server không tự register/fetch registry, actuator health/prometheus.
Discovery prod	discovery-server/src/main/resources/application-prod.yaml	DISCOVERY_PORT, EUREKA_INSTANCE_HOSTNAME, self-preservation, prod actuator detail.
Docker	compose.yaml	PostgreSQL, Keycloak, discovery, gateway, service env, database riêng, networks, healthcheck, optional profiles.
Security	lifebalance-security/.../LifebalanceSecurityAutoConfiguration.java	JWT decoder, audience validator, SecurityFilterChain, public actuator/status, stateless resource server.
Timeline internal API	timeline-service/.../TimelineTaskController.java	Endpoint /api/timeline/tasks yêu cầu X-Lifebalance-Internal-Secret.
Common	lifebalance-common/src/main/java/com/lifebalance/common	ApiResponse, ApiError, AppException, PageableLimits, GlobalExceptionHandler.
Service folders	identity/task/timeline/resource-capital/finance/notification/analytics/ai	Controller, DTO, domain/entity, repository, service, validation/integration theo từng bounded context.

35. Kết Luận Ngắn Gọn Để Chốt Khi Bảo Vệ

Backend LifeBalance được thiết kế theo hướng microservice có kiểm soát: Maven parent giúp quản lý 12 module trong một repo; Docker Compose giúp chạy nhiều service; Gateway là cửa vào; Eureka làm discovery; Keycloak/JWT xử lý xác thực; common/security chuẩn hóa phần dùng chung; mỗi service nghiệp vụ sở hữu domain và database logical riêng.

Điểm mạnh của thiết kế là ranh giới module rõ, dễ giải thích theo domain, có gateway/discovery/security/healthcheck, có test ở nhiều module và có hướng vận hành bằng Docker. Điểm cần cải tiến là consistency liên service, tracing, route/test contract gateway, cấu hình secret nội bộ và test coverage ở các service ít test. Những điểm này không phủ nhận kiến trúc mà là lộ trình nâng cấp hợp lý khi đưa hệ thống lên production lớn hơn.

Nếu cần trả lời thật ngắn trước hội đồng: “Em xây backend theo hướng microservice. Hệ thống có 12 module, gồm 2 module hạ tầng điều phối, 2 module dùng chung và 8 service nghiệp vụ. Client đi vào Gateway, Gateway dùng Eureka để định tuyến đến service. Security dùng Keycloak/JWT và module lifebalance-security. Dữ liệu được tách theo service ở mức database logical. Code trong service đi theo Controller-Service-Repository, dùng DTO, validation, exception handler và transaction cục bộ. Em đã review các rủi ro chính như discovery hostname, gateway route/test, internal secret và consistency liên service.”